



Pontificia Universidad
JAVERIANA
Cali

Facultad de Ingeniería y Ciencias
Departamento de Electrónica y Ciencias de la Computación

Carrera de Ingeniería Electrónica

Sistemas Digitales

Linux Sockets

Febrero 2022

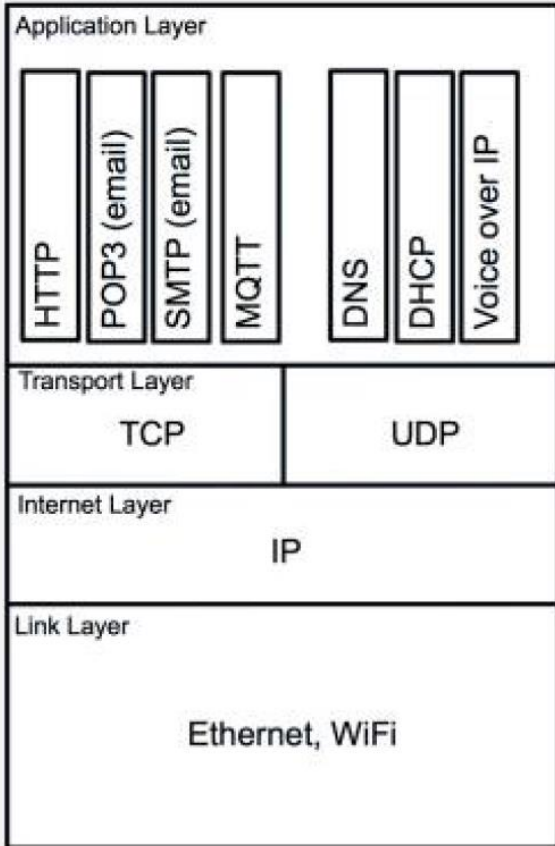
Tek



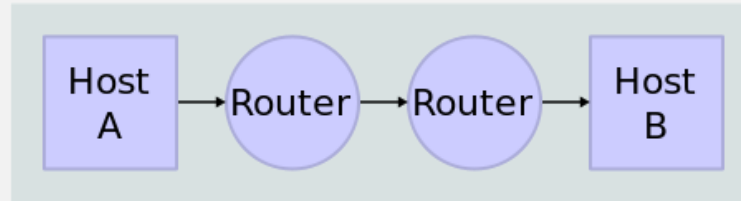
Sockets Interface

- What if communication across a network of computers is needed?
 - A process on one machine can use sockets to communicate with a process on another.
 - Sockets may also be used between processes on the same machine.
- What is a Socket?
 - A *socket* is a communication mechanism that allows client/server systems to be developed either locally, on a single machine, or across networks.

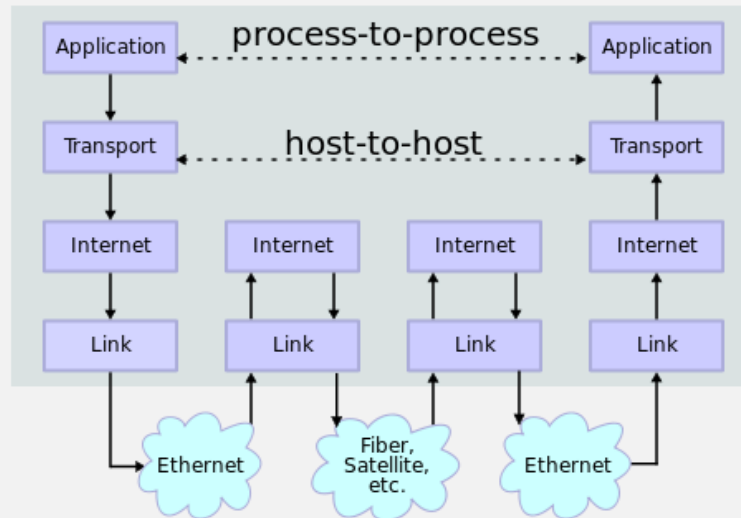
Internet Protocol Suite



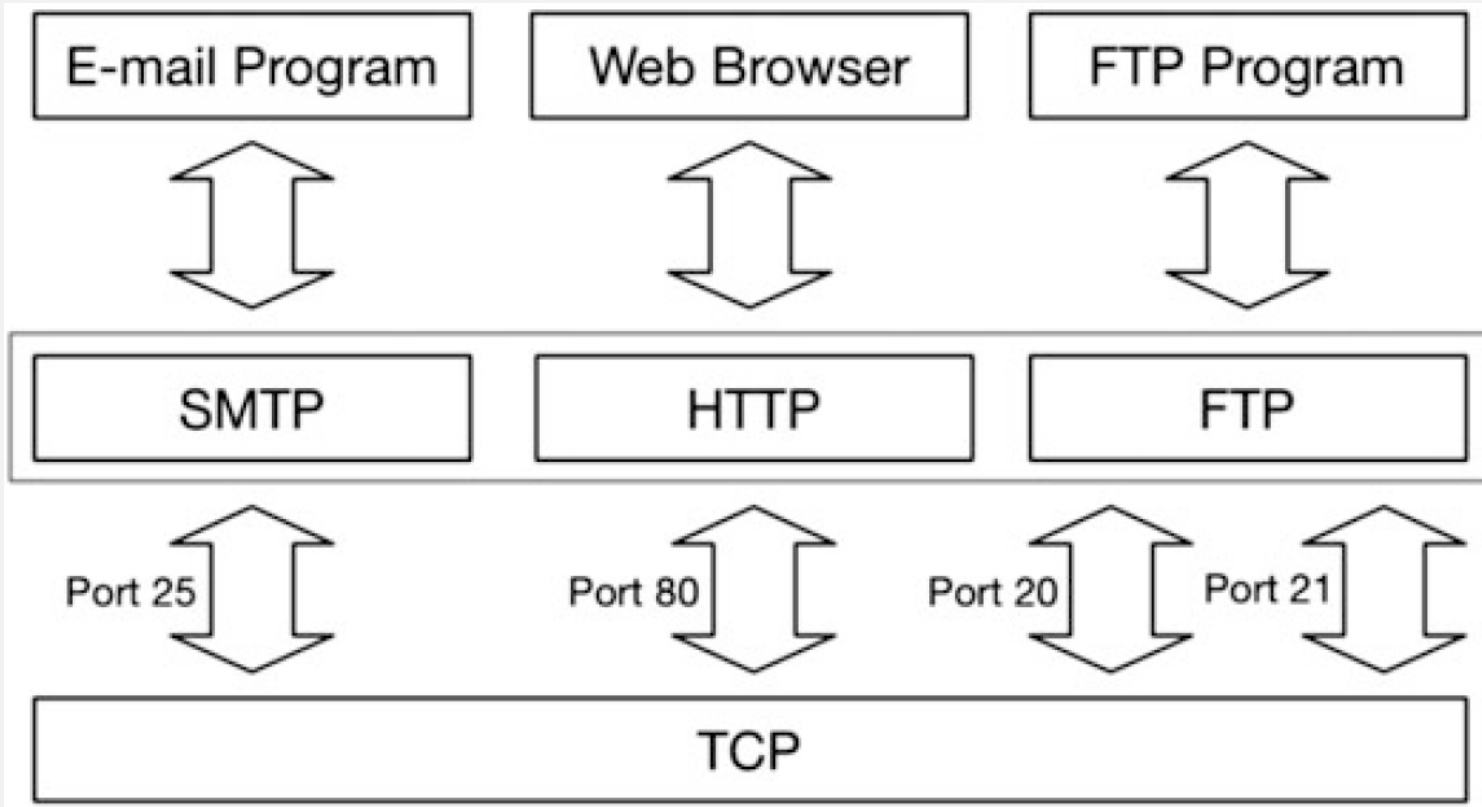
Network Topology



Data Flow



Applications using TCP

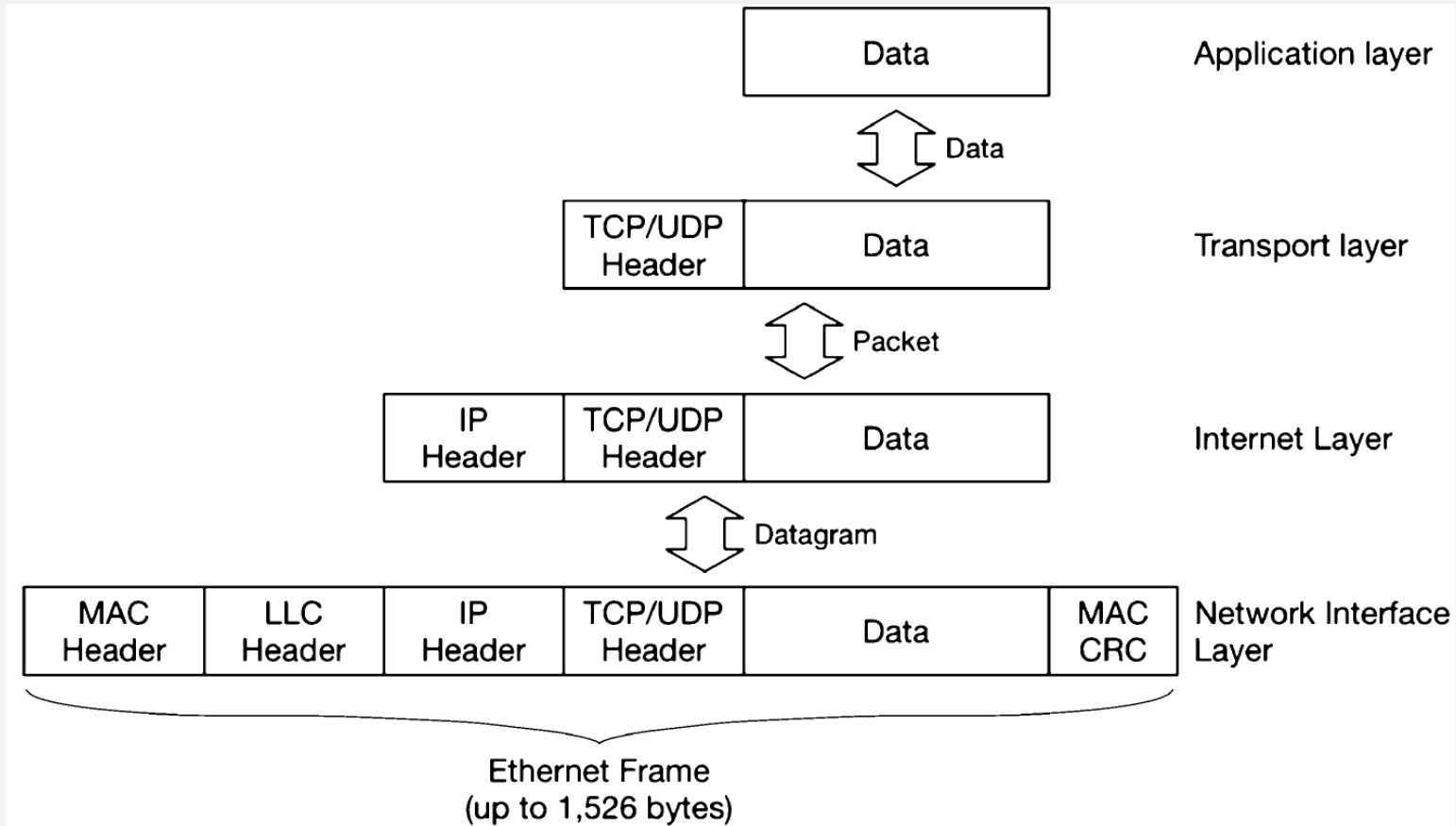


Application = (HostIP,
Protocol, PortNumber)

Application
Layer

Transport
Layer

Data Format in TCP/IP layers

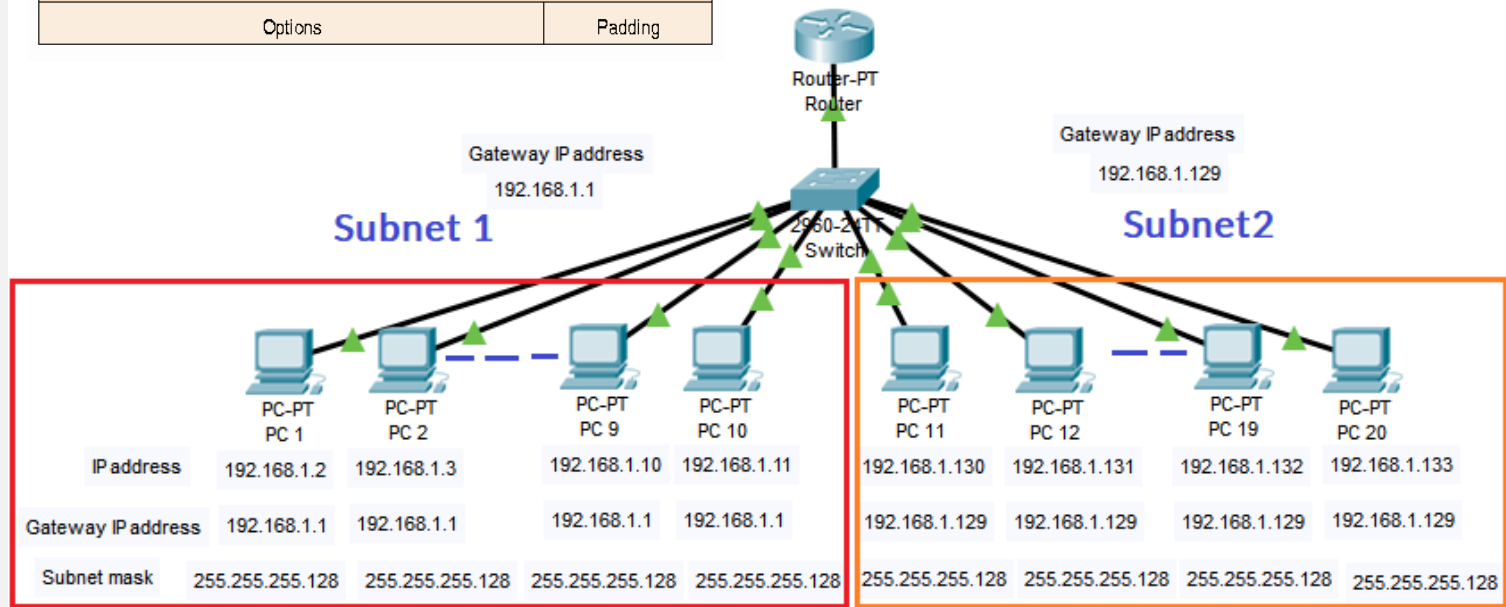


Internet Equipment

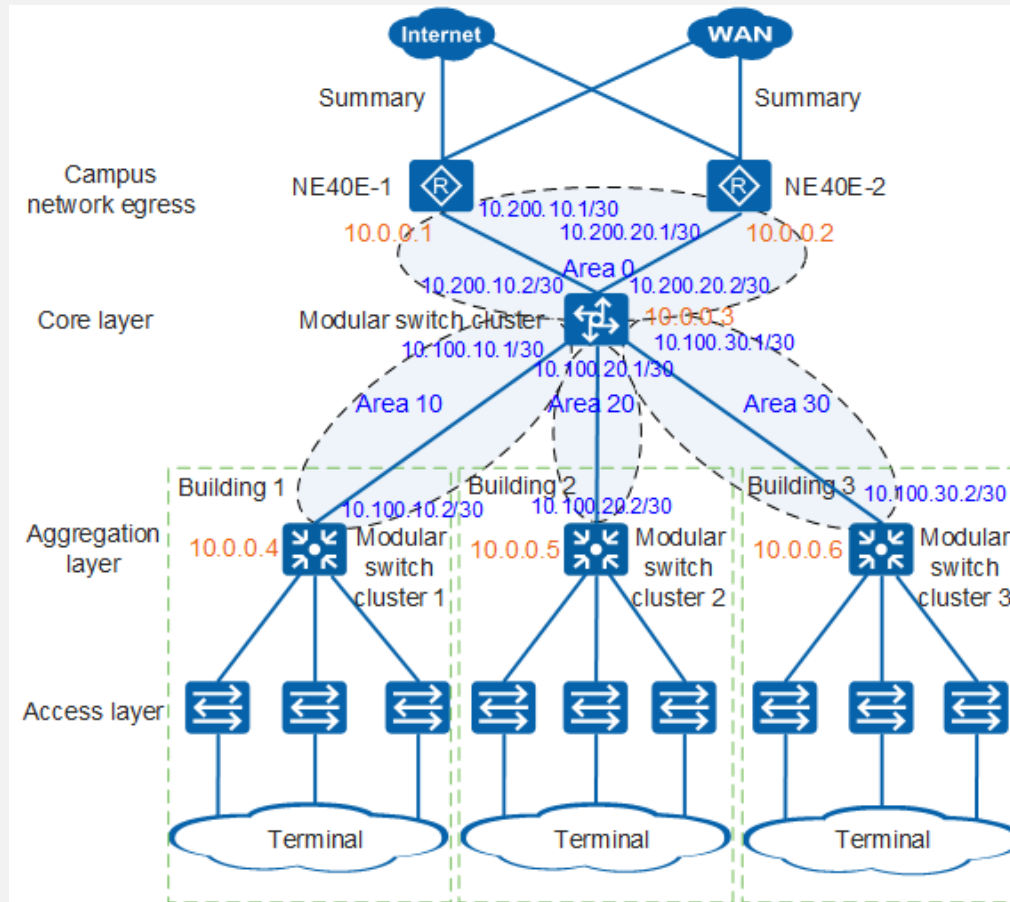


Internet Networking

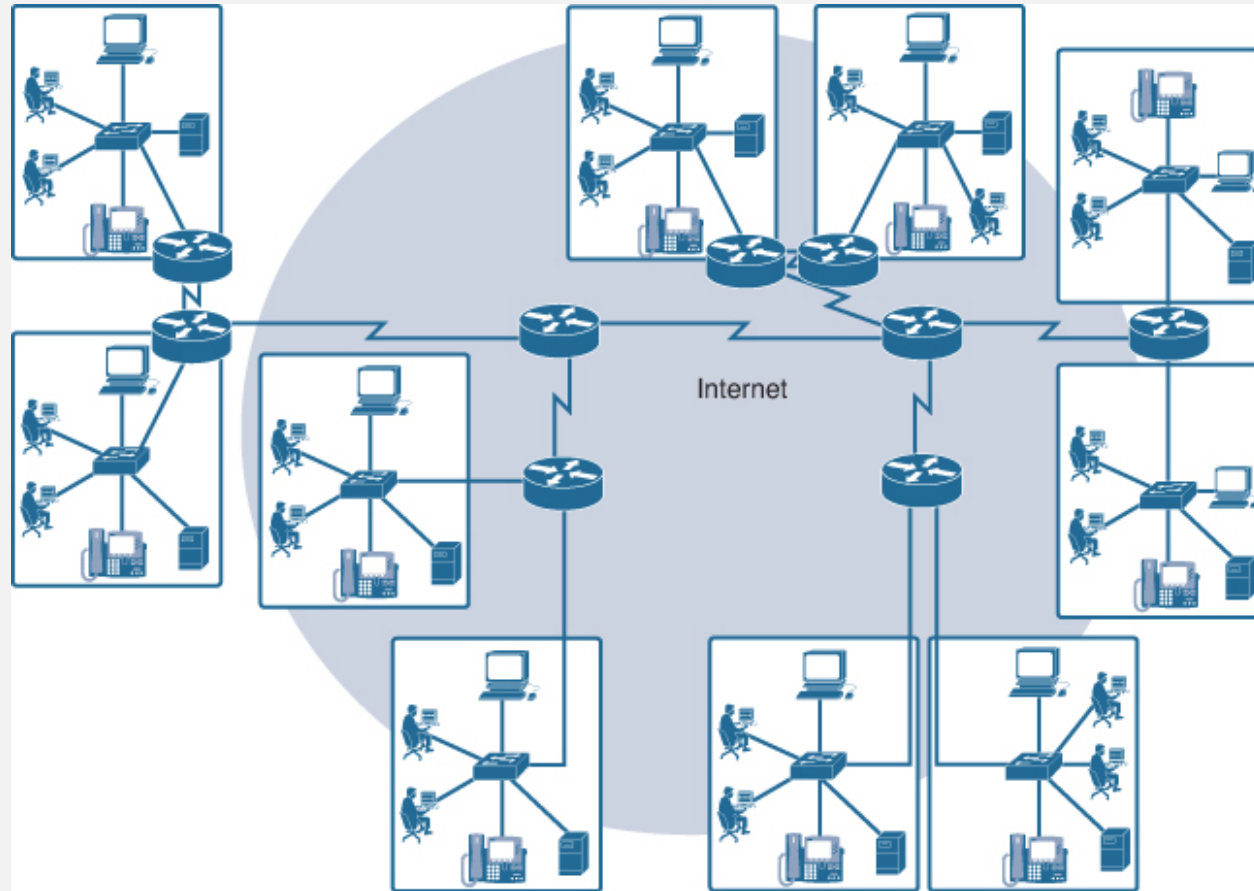
0	4	8	16	19	31
Version	IHL	Type of Service	Total Length		
Identification			Flags	Fragment Offset	
Time To Live		Protocol	Header Checksum		
Source IP Address					
Destination IP Address					
Options					Padding



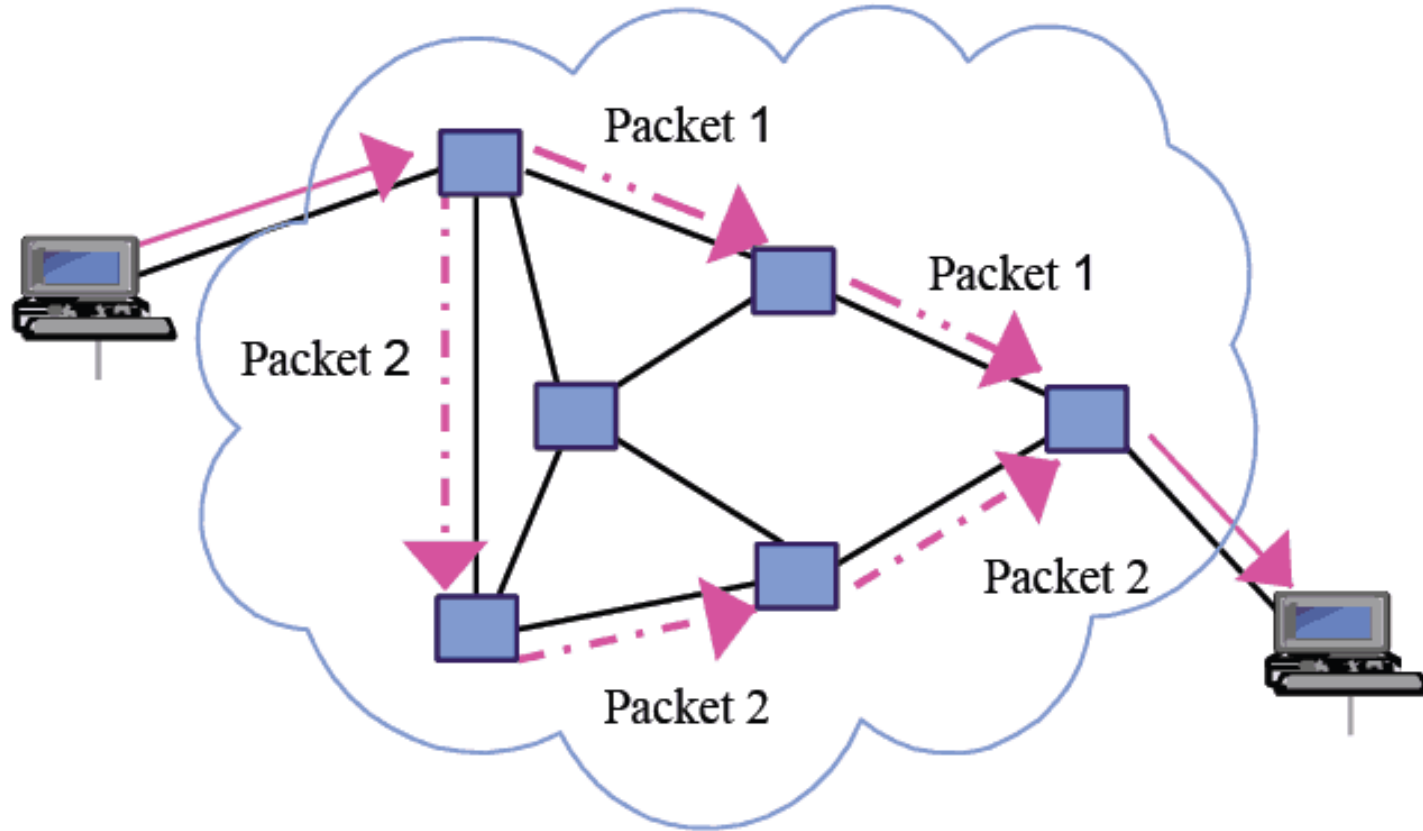
Internet Hierarchy



Internet Hierarchy



Internet Routing



Socket Connections

- The socket mechanism can implement multiple clients attached to a single server.
 - You can think of socket connections as telephone calls into a busy building:
 - A call comes into an organization and is answered by a receptionist who puts the call through to the correct department (the server process) and from there to the right person (the server socket: HostIP and PortNumber). Each incoming call (client) is routed to an appropriate end point and the intervening operators are free to deal with further calls.
-

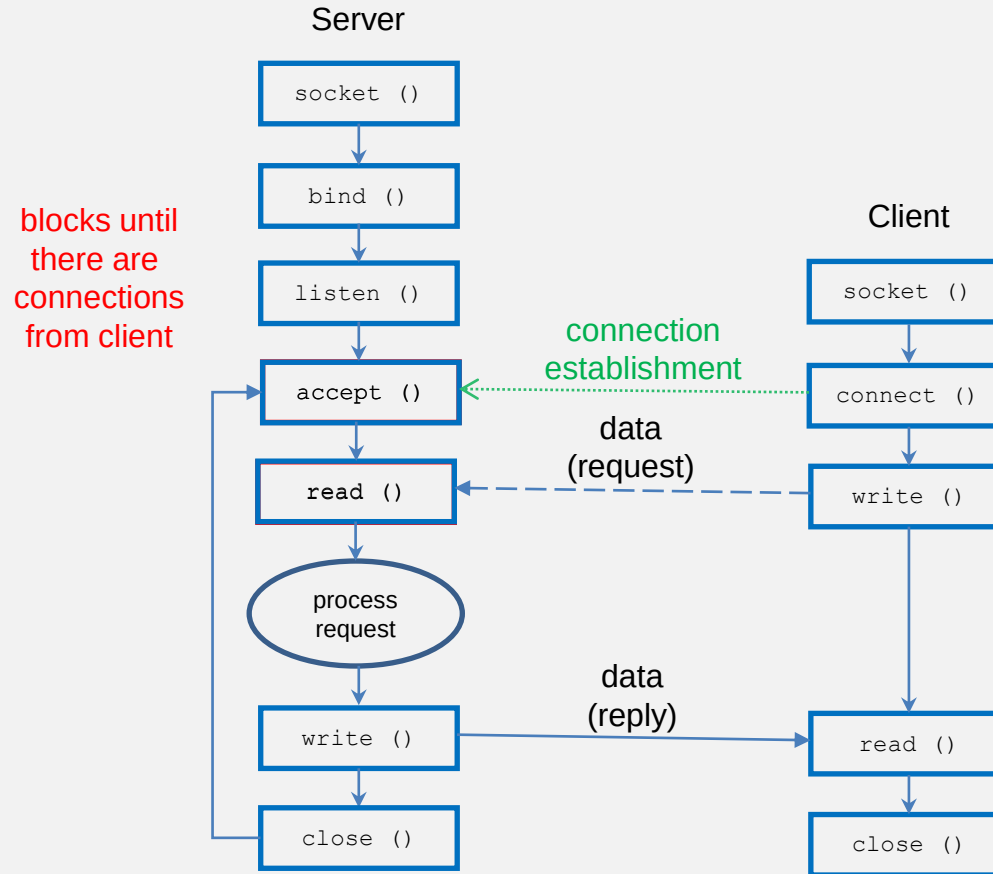
Socket Connections – Server side

1. A server application **creates a socket** using the system call `socket`.
2. Next, the server process **gives the socket a name** using the system call `bind`.
3. The server process then **waits for a client to connect** to the named socket. The system call, `listen`, creates a queue for incoming connections.
4. The server can **accept connections** using the system call `accept`.
 - When the server calls `accept`, a new socket is created that is **distinct** from the named socket. This new socket is used solely for communication with this particular client. The named socket remains open for further connections from other clients.

Socket Connections – Client side

1. The client `creates an unnamed socket` by calling `socket`.
2. It then calls `connect` to `establish a connection with the server` by using the server's named socket as an address.

Using Sockets



Creating a Socket - socket

- Include files

```
#include <sys/types.h>
#include <sys/socket.h>          /* stuff related to
                                   sockets */
```

- Function Prototype:

```
int socket ( int domain, int type, int protocol );
```

- Parameters:

- domain: typically AF_INET (UNIX network sockets)
- type: typically SOCK_STREAM (TCP/IP connection)
- protocol: typically 0 (default protocol)

- Returns:

Socket descriptor

AF_INET Socket Addresses

- Specified using a structure called `sockaddr_in`:

```
struct sockaddr_in
{
    short int          sin_family; /* AF_INET */
    unsigned short int sin_port;   /* Port number */
    struct in_addr     sin_addr;  /* Internet address */
};
```

- The IP address structure, `in_addr`, is defined as follows (IP v4):

```
struct in_addr
{
    unsigned long int s_addr; /* four bytes of an IP address
                               constitute a single 32-bit value */
};
```

Naming a Socket - bind

- Include files

```
#include <sys/socket.h>          /* stuff related to  
                                sockets */
```

- Function Prototype:

```
int bind ( int socket, const struct sockaddr *address,  
           size_t address_len );
```

- Parameters:

- socket: socket descriptor
- address: typically AF_INET Socket Address
- address_len: typically **sizeof** (**struct** sockaddr_in)

Creating a Socket Queue - listen

- Include files

```
#include <sys/socket.h>          /* stuff related to  
                                sockets */
```

- Function Prototype:

```
int listen ( int socket, int backlog );
```

- Parameters:

- socket: socket descriptor
- backlog: maximum number of pending connections that may be held in the queue; further connections will be refused

Accepting Connections - accept

- Include files

```
#include <sys/socket.h>          /* stuff related to  
                                   sockets */
```

- Function Prototype:

```
int accept ( int socket, struct sockaddr *address,  
             size_t *address_len );
```

- Parameters:

- socket: socket descriptor the client attempts to connect
- address: typically AF_INET Socket Address
- address_len: typically sizeof (struct sockaddr_in)

- Returns:

New socket descriptor

Requesting Connections - connect

- Include files

```
#include <sys/socket.h>          /* stuff related to  
                                sockets */
```

- Function Prototype:

```
int connect ( int socket, struct sockaddr *address,  
              size_t *address_len );
```

- Parameters:

- socket: socket descriptor
- address: typically AF_INET Socket Address
- address_len: typically sizeof (struct sockaddr_in)

- Return Value

If the connection or binding succeeds, 0 is returned. On error, -1 is returned, and `errno` is set appropriately.

Writing to a Socket - write

- Include files

```
#include <unistd.h>                /* provides access to the  
                                   POSIX operating system  
                                   API*/
```

- Function Prototype:

```
size_t write ( int sockdes, const void *buf, size_t nbyte );
```

- Parameters:

- sockdes: socket descriptor
- buf: pointer to buffer
- nbyte: number of bytes to write

- Returns:

the number of bytes actually written

Reading from a Socket - read

- Include files

```
#include <unistd.h>          /* provides access to the  
                             POSIX operating system  
                             API*/
```

- Function Prototype:

```
size_t read ( int sockdes, const void *buf, size_t nbyte );
```

- Parameters:

- sockdes: socket descriptor
- buf: pointer to buffer
- nbyte: number of bytes to read

- Returns:

the number of bytes actually read

Closing a Socket - close

- Include files

```
#include <unistd.h>
```

```
/* provides access to the  
   POSIX operating system  
   API*/
```

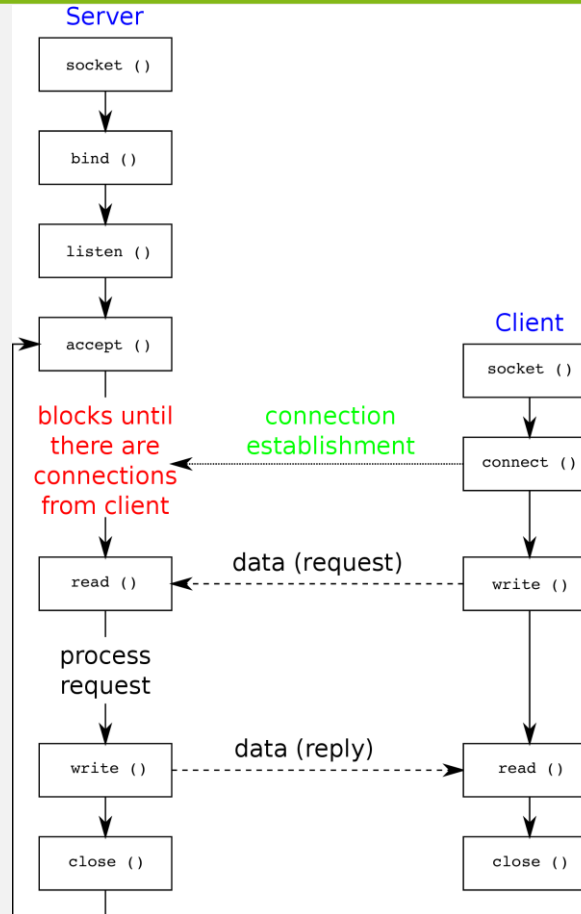
- Function Prototype:

```
int close ( int sockdes );
```

- Parameters:

- sockdes: socket descriptor

Using Sockets version 1



Using Sockets version 1

Simple server skeleton

```
/* Create an unnamed TCP/IP socket for the server. */
server_sockfd = socket ( AF_INET, SOCK_STREAM, 0 );

/* Name the socket allowing connections from any client. */

server_address.sin_family = AF_INET;
server_address.sin_addr.s_addr = htonl ( INADDR_ANY );
server_address.sin_port = htons ( SERVER_PORT );
server_len = sizeof ( server_address );
bind ( server_sockfd, (struct sockaddr *) &server_address, server_len );

/* Create a connection queue and wait for up to MAX_CLIENTS clients. */

listen ( server_sockfd, MAX_CLIENTS );
while ( 1 )
{
    char ch;

    printf ( "server waiting\n" );

    /* Accept a connection. */

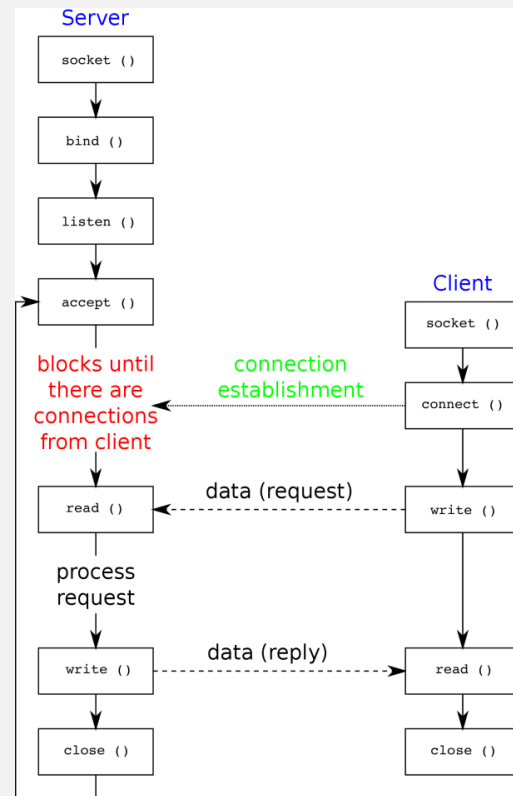
    client_len = sizeof ( client_address );
    client_sockfd = accept ( server_sockfd, (struct sockaddr *) &client_address,
                           (socklen_t *) &client_len );

    /* We can now read/write to client on client_sockfd. */

    read ( client_sockfd, &ch, 1 );
    ch++;
    write ( client_sockfd, &ch, 1 );

    /* Close connection. */

    close ( client_sockfd );
}
```



Using Sockets version 1

Client skeleton

```
/* Create an unnamed TCP/IP socket for the client. */
client_sockfd = socket ( AF_INET, SOCK_STREAM, 0 );

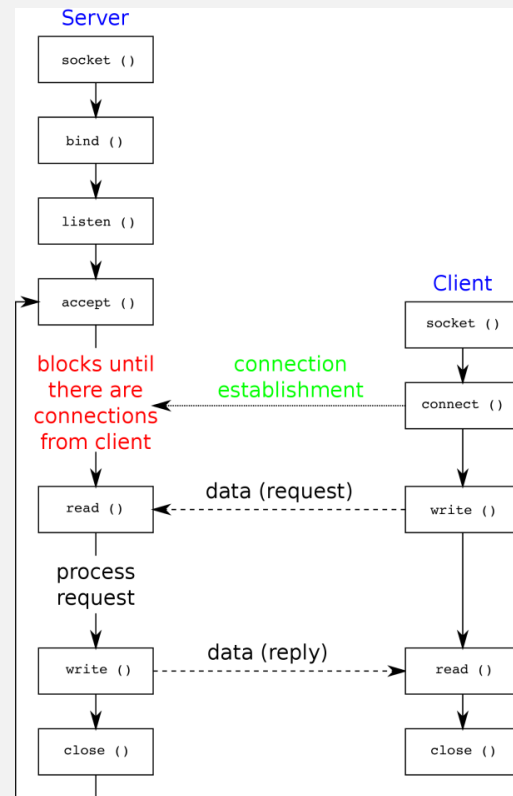
/* Name the socket, as agreed with the server. */
server_address.sin_family = AF_INET;
server_address.sin_addr.s_addr = inet_addr ( "127.0.0.1" );
server_address.sin_port = htons ( SERVER_PORT );
server_len = sizeof ( server_address );

/* Now connect our socket to the server's socket. */
result = connect (client_sockfd, (struct sockaddr *) &server_address,
                  server_len );

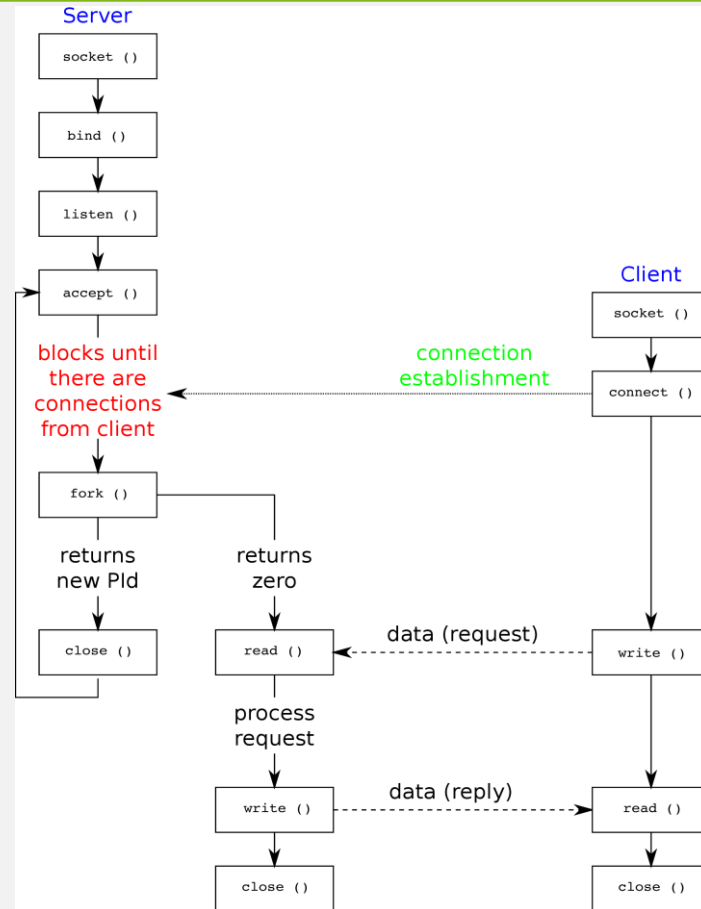
/* We can now read/write via sockfd. */

printf ( "client is ready to send... Please enter a char " );
fflush ( stdout );
fflush ( stdin );
scanf ( "%c", &ch );
write (client_sockfd, &ch, 1 );
read (client_sockfd, &ch, 1 );
printf ( "char from server = %c\n", ch );

/* Close connection. */
close (client_sockfd );
```



Using Sockets version 2



Using Sockets version 2

Server using fork - skeleton

```
/* Create a connection queue and wait for up to MAX_CLIENTS clients. */
listen ( server_sockfd, MAX_CLIENTS );

/* Ignore child exit details. */
signal ( SIGCHLD, SIG_IGN );

while ( 1 )
{
    char ch;

    printf ( "server waiting\n" );

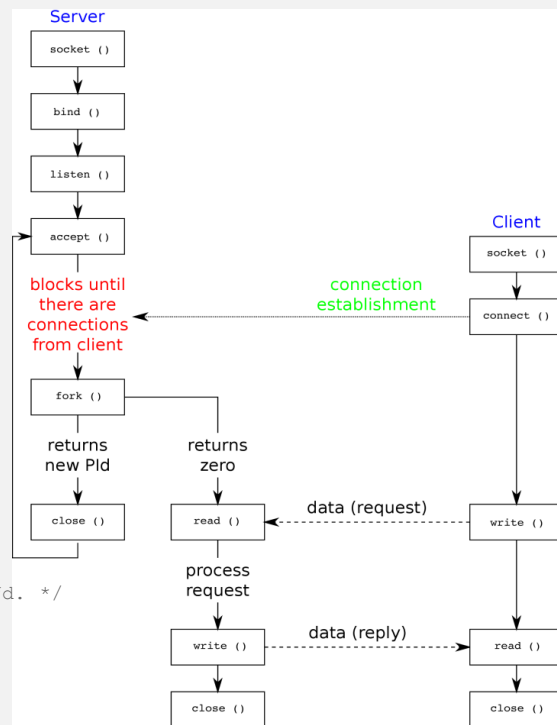
    /* Accept a connection. */

    client_len = sizeof ( client_address );
    client_sockfd = accept ( server_sockfd, (struct sockaddr *) &client_address,
                           (socklen_t *) &client_len );

    /* Fork to create a process for this client. */

    if ( fork () == 0 )
    {
        /* If we're the child, we can now read/write to the client on client_sockfd. */

        read ( client_sockfd, &ch, 1 );
        ch++;
        write ( client_sockfd, &ch, 1 );
        close ( client_sockfd );
        exit ( 0 );
    }
    else
        /* Otherwise, we must be the parent and our work for this client is finished. */
        close ( client_sockfd );
}
```



Getting Network Information

- Use host database functions:

```
#include <netdb.h>
struct hostent *gethostbyname ( const char *name );

struct hostent
{
    char *h_name;           /* name of the host */
    char **h_aliases;       /* list of aliases (nicknames) */
    int  h_addrtype;        /* address type */
    int  h_length;          /* length in bytes of the address */
    char **h_addr_list      /* list of address (network order) */
};
```

- The address list needs to be cast to the appropriate address type and converted from network ordering to a printable string:

```
#include <arpa/inet.h>
char *inet_ntoa ( struct in_addr in );
```

Using Sockets version 2

Client using network info - skeleton

```
int main ( int argc, char *argv[] )
{
    :
    char          *host,
                  **names,
                  **addrs;

    struct hostent *hostinfo;

    :

    host = argv [1];
    hostinfo = gethostbyname ( host );

    /* Create a TCP/IP socket for the client */

    sockfd = socket ( AF_INET, SOCK_STREAM, 0 );

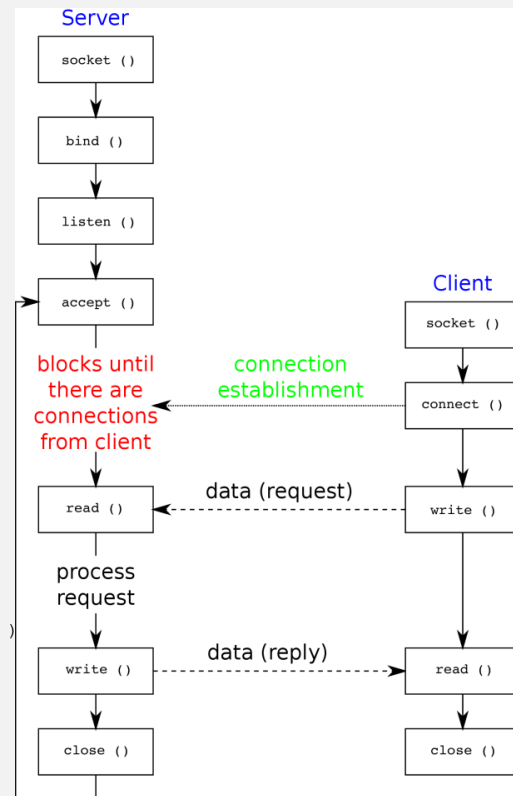
    /* Name the socket, as agreed with the server, using its first address */

    address.sin_family = AF_INET;
    addrs = hostinfo->h_addr_list;
    address.sin_addr.s_addr = inet_addr ( inet_ntoa ( *(struct in_addr *) *addrs ) );
    address.sin_port = htons ( SERVER_PORT );
    len = sizeof ( address );

    /* Now connect our socket to the server's socket. */

    result = connect ( sockfd, (struct sockaddr *) &address, len );

    /* We can now read/write via sockfd. */
}
```



Further Reading

MATTHEW, Neil; STONES, Rick. **Beginning Linux Programming, 4th Edition**. Wiley Publishing, 2008. **Chapters 15**.